

Low Level Design

Modules, algorithms, schema and API reference

Research Assistant - local-first multi-agent research workspace
Generated from 11-lld.md

11. Low Level Design

Table of contents

1. Module breakdown
2. Verification loop internals
3. Retrieval pipeline internals
4. Source adapters and discovery internals
5. Ingestion internals
6. Annotations and paper reader internals
7. Database schema
8. REST API reference
9. Error handling and degradation matrix
10. Testing strategy
11. Performance optimizations
12. Design vs. implementation notes

1. Module breakdown

1.1 Application startup and configuration

Symbol	Signature or shape	Responsibility
<code>create_app</code>	<code>def create_app() -> FastAPI</code> in <code>app\main.py</code>	Builds the FastAPI app, adds CORS, and mounts <code>/v1</code> routes.
<code>lifespan</code>	<code>async def lifespan(app: FastAPI)</code>	Configures logging, creates tables, adds legacy citation columns, runs startup note migrations, rehydrates the vector store, and closes Ollama on shutdown.
<code>Settings</code>	<code>class Settings(BaseSettings)</code> in <code>core\config.py</code>	Loads database, Ollama, source, web, and loop settings from <code>.env</code> .
<code>configure_sqlite_pragmas</code>	<code>def configure_sqlite_pragmas(async_engine,</code>	Registers SQLAlchemy connect hook for <code>foreign_keys=ON</code> , <code>WAL</code> , and <code>busy_timeout=5000</code> .
<code>run_startup_migrations</code>	<code>async def run_startup_migrations(conn) -> None</code>	Adds <code>notes.chunk_id</code> , <code>anchor_quote</code> , <code>anchor_start</code> , <code>anchor_end</code> , and <code>color</code> if missing.

1.2 API modules

File	Functions
<code>api\v1\health.py</code>	<code>health</code> , <code>models</code>
<code>api\v1\spaces.py</code>	<code>create_space</code> , <code>list_spaces</code> , <code>get_space</code> , <code>rename_space</code> , <code>archive_space</code> , <code>unarchive_space</code> , <code>duplicate_space</code> , <code>delete_space</code> , <code>search_content</code> , <code>list_turns</code>

File	Functions
api\v1\search.py	search_tags, search, source_adapters
api\v1\papers.py	pin_paper, capture_web, get_paper, paper_content, paper_status, unpin
api\v1\chat.py	chat
api\v1\compare.py	compare, list_comparisons, get_comparison
api\v1\notes.py	create_note, auto_note, list_notes, paper_annotations, get_note, update_note, delete_note, get_memory
api\v1\verification.py	verification_trail

1.3 Services and agents

Component	Main public methods	Notes
DiscoveryService	discover(self, topic, filters=None)	Plans, fans out, normalizes, deduplicates, ranks, and partitions web results.
PaperService	pin_and_ingest, capture_web, get, get_status, unpin	Handles persistence, SSRF-safe fetches, parsing, chunking, embedding, and vector registration.
QAOrchestrator	answer(self, space_id, question)	Builds memory, rewrites queries, retrieves evidence, runs verification, persists turns and trails.
CompareService	compare, list_reports, get_report	Validates pins, builds profiles, generates matrix and gaps, verifies conclusions, caches reports.
NoteService	auto_generate, create, update, delete, list, annotations, augment_evidence_with_notes	Implements notebook, auto notes, reader annotations, and note evidence chunks.
QueryPlanner.plan	async def plan(self, topic: str, filters: SearchFilters None = None, include_web: bool = False, domain_tags: list[str] None = None, limit: int = 20) -> SearchPlan	Parses tags and creates source hints.
RankingAgent.rank	async def rank(self, papers, topic: str, domain_tags: list[str] None = None)	Keyword overlap, citation count, OA bonus, domain boost, and web penalty.

Component	Main public methods	Notes
<code>AnswerAgent.generate</code>	<pre>async def generate(self, question: str, evidence: EvidenceSet, feedback_history=None) -> Draft</pre>	Uses Ollama medium model at temperature 0.3 and extracts bracket citations into claims.
<code>PeerReviewerAgent.review</code>	<pre>async def review(self, question, evidence, draft, iteration: int = 1) -> ReviewVerdict</pre>	Uses verify model at temperature 0.0 and large model on iteration 3 or later.

2. Verification loop internals

2.1 Data structures

`schemas\chat.py` defines `ScoredChunk`, `EvidenceSet`, `ConversationContext`, `Claim`, `Draft`, `CitationDTO`, `TurnState`, and `AnsweredTurn`. `TurnState` exists as a Pydantic model but the delivered driver does not use a LangGraph state machine. Runtime state is local variables in `VerificationLoopDriver.run`.

`schemas\verification.py` defines `ClaimStatus` values SUPPORTED, PARTIAL, UNSUPPORTED, CONTRADICTED, MISCITED; `VerdictType` values APPROVED, REVISE, EVIDENCE_GAP, REJECT; `LoopAction` values ACCEPT, REGENERATE, RE_RETRIEVE, ABSTAIN, FINALIZE_BEST_EFFORT; plus `ClaimFinding`, `ReviewVerdict`, and `LoopTrace`.

2.2 Loop driver

`VerificationLoopDriver(retriever, answer_agent, reviewer_agent, controller, trail=None)` exposes `async def run(self, space_id: str, question: str, turn_id: str | None = None, extra_queries: list[str] | None = None)`.

Algorithm:

1. Allocate `turn_id`.
2. Retrieve initial evidence inside `asyncio.timeout(self._remaining_seconds(start))`.
3. Generate a draft with `AnswerAgent.generate(question, evidence, history)`.
4. Review with `PeerReviewerAgent.review(question, evidence, draft, iteration)`.
5. Normalize the verdict with `LoopPolicies.prepare_verdict`.
6. Keep the best draft and its original evidence by highest reviewer score.
7. Ask `RefinementController.decide` for the next action.
8. Persist traces on terminal action.
9. Return `_answered`, `_finalize_best_effort`, or `_abstain`.

2.3 Acceptance rules and citation validation

`LoopPolicies.is_acceptable` requires threshold score, verdict APPROVED, no UNSUPPORTED, CONTRADICTED, or MISCITED claims, and reviewer coverage for every draft claim. `prepare_verdict` aligns reviewer findings to draft claims by `claim_id`, or by position if counts match, or forces REVISE when the reviewer omitted claims.

`LoopPolicies._resolve_cited_id` checks each cited chunk against current evidence. It repairs only recognizable shorthand such as `chunk1`, `chunk-1`, `c1`, numeric position, or a unique substring. Arbitrary fabricated IDs become MISCITED.

2.4 ConvergenceGuard and LC mapping

The implemented guard is `Convergence(min_improvement=0.05).has_stalled(history)` returns true after three verdicts if both recent score gains are below 0.05.

Rule	Code mapping	Notes
LC-1 Iteration budget	<code>LoopPolicies.max_iterations</code> , checked in <code>decide</code> .	Configured by <code>LOOP_MAX_ITERATIONS</code> .
LC-2 Acceptance threshold	<code>is_acceptable</code> .	Needs score, APPROVED, and no bad claims.
LC-3 Routing	<code>LoopPolicies.decide</code> .	UNSUPPORTED re-retrieves; PARTIAL, MISCITED, CONTRADICTED, or REVISE regenerates; REJECT abstains.
LC-4 Feedback carry-forward	<code>history</code> passed to <code>AnswerAgent.generate</code> .	Prompt receives prior verdicts.
LC-5 Convergence guard	<code>Convergence.has_stalled</code> .	Stops with <code>FINALIZE_BEST_EFFORT</code> .
LC-6 Exhaustion behavior	<code>_finalize_best_effort</code> .	Keeps SUPPORTED and PARTIAL claims only.
LC-7 Abstention	<code>_abstain</code> .	Returns <code>Not answerable from the attached papers</code> .
LC-8 Independence	Separate answer and reviewer prompts/models.	Reviewer temperature is 0.0.
LC-9 Determinism	Agent temperatures in code.	Generator 0.3, reviewer 0.0.
LC-10 Auditability	<code>VerificationTrail.persist</code> plus <code>verification_iterations</code> .	UI displays trail.
LC-11 Latency guard	<code>asyncio.timeout(self._remaining_seconds)</code> uses <code>LOOP_WALL_CLOCK_CAP_MS</code> .	
LC-12 Escalation	<code>PeerReviewerAgent.review</code> uses large model when <code>iteration >= 3</code> .	Final adjudication path.

2.5 Finalizers, timeout, and persistence

`_answered` builds unique `CitationDTO` entries from surviving claims and evidence metadata. `_finalize_best_effort` keeps only SUPPORTED and PARTIAL claims, flags low confidence, or abstains when no claims survive. `_abstain` returns a non-answer and optional evidence titles.

Initial retrieval failures return an abstention with a synthetic REVISE verdict. Failures after a best draft exists return best-effort with warning. `LLMUnavailableError` is raised by `OllamaClient.chat`, `chat_stream`, and `embed` wrappers.

`QAOrchestrator.answer` writes user and assistant turns, non-note citations, and all loop traces to `verification_iterations`.

3. Retrieval pipeline internals

`QueryRewriter.rewrite(question, conversation_context)` only rewrites when referential terms are present. It uses the last two recent turns, each truncated to 200 characters, and returns `[standalone, original]` when the rewrite differs. This replaced a failed naive approach where embedding a full memory blob biased retrieval toward stale topics.

`HybridRetriever.retrieve(space_id, question, scope=None, extra_queries=None)` builds a unique query list. Dense search embeds each query with Ollama `nomic-embed-text` and searches `InMemoryVectorStore` top 50. Sparse search scores chunks with TF-IDF-like term frequency and inverse document frequency. `ReciprocalRankFusion` combines rankings; `Reranker` adds lexical overlap; `MMR` selects diverse top 8 chunks; `ContextBuilder.pack(word_budget=1800, max_chunks=8)` produces the evidence set. The conservative budget prevents Ollama context overflow and empty drafts.

`NotesAwareRetriever` appends up to three relevant note chunks after paper evidence. Note chunks have IDs like `note-{note_id}` and are returned to the model but not inserted into `citations` rows because they have no chunk foreign key.

4. Source adapters and discovery internals

`SourceAdapter` declares `name`, `search(SearchQuery)`, `fetch_by_id(PaperIdentifier)`, `resolve_fulltext(RawPaperRecord)`, and `health_check`. Only `search` is implemented by live adapters.

Adapter	Implementation detail
<code>ArxivAdapter</code>	Atom API; adds <code>cat:</code> fragments from domain tags; captures categories and PDF link.
<code>OpenAlexAdapter</code>	Works API; normalizes DOI, returns citation count, OA status, and PDF URL.
<code>CrossrefAdapter</code>	Works API; extracts title, authors, year, DOI, venue, and reference count.
<code>PubMedAdapter</code>	NCBI <code>esearch</code> then <code>esummary</code> ; optional API key; returns PMID, DOI, authors, journal, and year.
<code>SemanticScholarAdapter</code>	Graph API; requires <code>SEMANTIC_SCHOLAR_API_KEY</code> to avoid unauthenticated rate limits.
<code>COREAdapter</code>	CORE v3; returns empty list unless <code>CORE_API_KEY</code> is configured.
<code>WebSearchAdapter</code>	Selected web provider; caps to 8 results; returns source <code>web</code> records with URL and snippets.

`SourceRegistry.fan_out` wraps each adapter in a `CircuitBreaker` and `asyncio.timeout(settings.source_timeout_s)`. This is needed because httpx operation timeouts do not guarantee a whole-call deadline for slow trickle responses.

`sources\taxonomy.py` defines tags such as `ml`, `nlp`, `medical`, `bio`, `physics`, `math`, `security`, `systems`, `climate`, and `econ`. `parse_tags` strips known tags and `plan_for_tags` derives source priority, arXiv categories, and query terms.

`Deduplicator.merge` uses DOI, arXiv, and PMID indexes first, then fuzzy title matching only after cheap gates: length, year, first-author surname, and token overlap. This changed the hot path from broad $O(n^2)$ fuzzy comparisons to about 2 ms for typical batches.

5. Ingestion internals

`validate_public_https_url(url)` rejects missing host, non-HTTPS scheme, loopback, private, link-local, multicast, reserved, unspecified, and invalid IPs. `PaperService` validates original URLs and final redirect URLs.

Limits and parsing:

- `MAX_PDF_BYTES = 30 * 1024 * 1024`.
- `MAX_PDF_PAGES = 300`.
- `MAX_WEB_CAPTURE_BYTES = 5 * 1024 * 1024`.
- PDFs must start with `%PDF-`.
- Full web capture checks robots.txt and supports HTML or PDF; selected-content capture skips network fetch.
- `_parse_pdf_pages(data)` uses `pypdf.PdfReader`.
- `_chunk_pages(pages, size=350, overlap=50)` never spans pages and carries page and heuristic section labels.

`OllamaClient.embed` posts to `/api/embed` in batches of 32. `_validate_vectors` requires matching vector count, non-empty vectors, and one shared vector dimension. If validation fails, chunks are not partially published.

`_upsert_paper_row` dedupes by canonical DOI, canonical arXiv ID, content hash, or web URL. `_load_chunks` reuses existing chunks and embeddings. `pin_and_ingest` checks the pin still exists before and after chunk persistence so an unpin during ingest does not publish stale chunks.

Statuses are QUEUED, FETCHING, PARSING, CHUNKING, EMBEDDING, PROFILING, READY, FAILED, and DEGRADED. Abstract-only ingest is DEGRADED. Exceptions mark FAILED.

6. Annotations and paper reader internals

The delivered `notes` table includes `chunk_id`, `anchor_quote`, `anchor_start`, `anchor_end`, and `color`. `run_startup_migrations` adds those columns idempotently.

The React `PaperReader` computes offsets with `textOffset(root, node, offset)` against the paragraph marked `data-chunk-body`, not the enclosing section. This avoids counting the section/page label and shifting anchors.

`ReaderChunk` sorts anchored notes by `anchor_start`, clips each range against the current cursor, and renders the visible remainder as a clickable highlight. Fully contained nested highlights may have no visible remainder; those notes remain accessible in the Notes tab.

7. Database schema

Schema verified from `backend\data\dev.db` using `PRAGMA table_info`.

7.1 Core tables

Table	Columns
<code>chunks</code>	<code>id</code> VARCHAR PK, <code>paper_id</code> VARCHAR FK, <code>section</code> VARCHAR, <code>page</code> INTEGER, <code>ordinal</code> INTEGER, <code>text</code> TEXT, <code>token_count</code> INTEGER, <code>vector_id</code> VARCHAR, <code>embedding_json</code> JSON
<code>citations</code>	<code>id</code> VARCHAR PK, <code>turn_id</code> VARCHAR FK, <code>chunk_id</code> VARCHAR FK, <code>claim_text</code> TEXT, <code>ordinal</code> INTEGER, <code>paper_id</code> VARCHAR, <code>section</code> VARCHAR, <code>page</code> INTEGER, <code>quote</code> TEXT

Table	Columns
comparison_reports	id VARCHAR PK, space_id VARCHAR FK, paper_ids JSON, matrix JSON, commonalities JSON, contradictions JSON, gaps JSON, confidence FLOAT, generated_at DATETIME
notes	id VARCHAR PK, space_id VARCHAR FK, paper_id VARCHAR FK, content TEXT, source VARCHAR, created_at DATETIME, updated_at DATETIME, chunk_id VARCHAR FK, anchor_quote TEXT, anchor_start INTEGER, anchor_end INTEGER, color VARCHAR
paper_profiles	id VARCHAR PK, paper_id VARCHAR FK UNIQUE, problem TEXT, method TEXT, dataset TEXT, metrics TEXT, results TEXT, limitations TEXT, future_work TEXT, generated_at DATETIME, verified BOOLEAN
papers	id VARCHAR PK, doi VARCHAR UNIQUE, arxiv_id VARCHAR UNIQUE, pmid VARCHAR, openalex_id VARCHAR, title TEXT, authors JSON, year INTEGER, venue VARCHAR, abstract TEXT, citation_count INTEGER, oa_status VARCHAR, pdf_url VARCHAR, source VARCHAR, content_hash VARCHAR UNIQUE, ingest_status VARCHAR, raw_payload JSON, created_at DATETIME
pins	id VARCHAR PK, space_id VARCHAR FK, paper_id VARCHAR FK, pinned_at DATETIME; unique (space_id, paper_id)
research_spaces	id VARCHAR PK, user_id VARCHAR FK, name VARCHAR, status VARCHAR, created_at DATETIME, updated_at DATETIME
space_memory	space_id VARCHAR PK/FK, rolling_summary TEXT, findings JSON, open_questions JSON, updated_at DATETIME
turns	id VARCHAR PK, space_id VARCHAR FK, role VARCHAR, content TEXT, created_at DATETIME
users	id VARCHAR PK, email VARCHAR UNIQUE INDEX, password_hash VARCHAR, created_at DATETIME
verification_iterations	id VARCHAR PK, turn_id VARCHAR FK, iteration INTEGER, draft_text TEXT, verdict VARCHAR, overall_score FLOAT, claim_findings JSON, missing_evidence_queries JSON, action_taken VARCHAR, latency_ms INTEGER, model_used VARCHAR, created_at DATETIME

Application connections set `PRAGMA foreign_keys=ON`, `journal_mode=WAL`, and `busy_timeout=5000`. A raw standalone SQLite connection may show `foreign_keys=0` because that pragma is connection-local.

8. REST API reference

Generated from live <http://localhost:8321/openapi.json>.

Method	Path	Request	Response	Errors
GET	/v1/admin/health	None	Health summary	200
GET	/v1/admin/models	None	Model name map	200
GET	/v1/search/tags	None	List of domain tags	200
POST	/v1/spaces	SpaceCreate { name }	Space summary	422 validation
GET	/v1/spaces	None	Space list	200
GET	/v1/spaces/{space_id}	Path space_id	Space detail with pins	404, 422
PATCH	/v1/spaces/{space_id}	SpaceUpdate { name }	Space detail	404, 422
DELETE	/v1/spaces/{space_id}	Path space_id	Delete result	404, 422
POST	/v1/spaces/{space_id}	Path space_id	Space summary	404, 422
POST	/v1/spaces/{space_id}	Path space_id	Space summary	404, 422
POST	/v1/spaces/{space_id}	Optional SpaceDuplicateRequest { copy_notes, name }	Space summary	404, 422
GET	/v1/spaces/{space_id}/turns	Query limit default 100	Turns with citations	404, 422
GET	/v1/spaces/{space_id}	Path space_id	Space memory	404, 422
GET	/v1/spaces/{space_id}	Query qch-content	Matching notes and turns	404, 422
POST	/v1/spaces/{space_id}	SearchQuery	RankedPaperList with results, web_results, source_health	422 validation
POST	/v1/spaces/{space_id}	RawPaperRecord	IngestJob	404, 422
DELETE	/v1/spaces/{space_id}/papers/{paper_id}	Path Ids	Unpin result	422
POST	/v1/spaces/{space_id}	WebCaptureRequest { url, title, content }	IngestJob	422 or capture validation errors

Method	Path	Request	Response	Errors
GET	/v1/papers/{paper_id}	Path paper_id	Paper DTO or detail not found object	422
GET	/v1/papers/{paper_id}	Path paper_id, optional space_id, limit 1 to 500	Paper and ordered chunks	404, 422
GET	/v1/papers/{paper_id}	Path paper_id	{ paper_id, status }	404, 422
POST	/v1/spaces/{space_id}	ChatRequest { question }	AnsweredTurn	404, 422, LLM degraded as low-confidence result
GET	/v1/turns/{turn_id}	Path turn_id	Verification iterations	422
POST	/v1/spaces/{space_id}	Query refresh, body CompareRequest { paper_ids, refresh }	Comparison report	404, 422, timeout
GET	/v1/spaces/{space_id}	Path space_ids	Comparison summaries	404, 422
GET	/v1/comparisons/{report_id}	Path report_id	Comparison report	404, 422
POST	/v1/spaces/{space_id}	NoteCreate	Note DTO	404, 422
GET	/v1/spaces/{space_id}/notes	Optional query paper_id	Notes	404, 422
POST	/v1/spaces/{space_id}/notes/{note_id}	Query refresh, paper_id	Note DTO	404, 422
GET	/v1/spaces/{space_id}/papers/{paper_id}/anchored notes	Path IDs	Anchored notes	404, 422
GET	/v1/notes/{note_id}	Path note_id	Note DTO	404, 422
PUT	/v1/notes/{note_id}	NoteUpdate { content }	Note DTO	404, 422
DELETE	/v1/notes/{note_id}	Path note_id	{ ok: true }	404, 422

8.1 Request schemas from OpenAPI

Schema	Required fields	Optional and default fields
ChatRequest	question: string	None
CompareRequest	paper_ids: array	refresh: boolean = false

Schema	Required fields	Optional and default fields
NoteCreate	None	paper_id, content = "", chunk_id, anchor_quote, anchor_start, anchor_end, color
NoteUpdate	None	content = ""
RawPaperRecord	source, title	authors, year, venue, abstract, doi, arxiv_id, pmid, openalex_id, citation_count = 0, oa_status, pdf_url, url, raw_payload
SearchFilters	None	year_min, year_max, sources, open_access_only = false, venue, author, min_citations
SearchQuery	query	filters, limit = 20 range 1 to 100, include_web = false, domain_tags, arxiv_categories, query_terms
SpaceCreate	name	None
SpaceUpdate	name minimum length 1	None
SpaceDuplicateRequest	None	copy_notes = false, name
WebCaptureRequest	url	title, content

9. Error handling and degradation matrix

Failure	Handling
Source timeout	SourceRegistry marks source DEGRADED and returns other sources.
Circuit open	Source marked UNAVAILABLE.
Missing Semantic Scholar key	Adapter raises skip error; registry reports degraded; search continues.
Unsafe PDF or web URL	Rejects or returns no PDF text; capture route raises validation error for explicit capture.
Oversized PDF or page	Rejects oversized download.
Non-PDF content at PDF URL	Rejected by magic byte check.
Robots disallow full web capture	Raises blocked by robots.txt for this user agent.
Embedding count or dimension mismatch	Raises before publishing chunks; ingest returns FAILED.

Failure	Handling
Pin removed during ingest	Ingest avoids publishing to runtime store and reports race-safe message.
Ollama unavailable during chat	Returns abstention or best-effort result with low-confidence warning.
Reviewer malformed JSON	Converts to REVISE verdict with feedback asking for shorter factual sentences.
Reviewer omits claims	<code>prepare_verdict</code> forces REVISE until every claim is covered.
SQLite locked	<code>retry_sqlite_locked</code> retries write operations.
Note anchor invalid	422 for missing content and quote, missing chunk, chunk-paper mismatch, or <code>anchor_end < anchor_start</code> .
Compare with fewer than two papers	422.
Compare paper not pinned	404 with missing paper IDs.

10. Testing strategy

The backend currently has 85 tests across 17 files. They cover:

- App startup smoke test.
- Domain tags, source routing, PubMed, arXiv category fragments, DuckDuckGo parsing, web provider selection, and backend avoidance of Google Scholar fetching.
- Source fan-out deadline and deduplicator correctness and performance.
- Persistence, canonical DOI/arXiv dedupe, rehydration, web capture, unsafe URL rejection, abstract-only degraded ingest, status endpoint, foreign-key configuration, citations, and verification trail persistence.
- Query rewriting regressions, including standalone pronoun rewrite and avoiding long memory blobs.
- Retrieval fusion, duplicate handling, extra query dense search, and context builder budget.
- Verification loop acceptance, revise then approve, re-retrieve, exhaustion, unsupported claim stripping, reject abstention, timeout, LLM unavailable warning, citation metadata, and evidence consistency for best draft.
- Notes, auto notes, note-context chunks, memory failure swallowing, space duplicate/delete/search, and FR-5/FR-6 routes.
- Compare profile persistence, tolerant JSON, validation, cache refresh, fixed-evidence verification, newest-report cache, and deterministic fallbacks.
- Annotation migration, content ordering, anchored note validation, and annotation ordering.

Notable regression tests:

Test	Bug locked in
<code>test_rewrite_query_stays_short_and_excludes_memory_blobs</code>	Prevents retrieval from embedding entire memory context.
<code>test_context_builder_bounds_prompt_size</code>	Prevents oversized prompts causing empty local drafts.
<code>test_best_draft_keeps_its_own_evidence_for_citations</code>	Ensures citations are validated against the evidence that produced the draft.

Test	Bug locked in
<code>test_embedding_count_mismatch_fails_without_partial_publish</code>	Prevents partial chunk publication after bad embedding responses.
<code>test_fan_out_enforces_per_source_deadline</code>	Ensures source latency is bounded outside httpx operation timeouts.
<code>test_deduplicator_merges_variants_and_stays_fast</code>	Protects identifier indexes and fuzzy-match gates.
<code>test_backend_never_fetches_google_scholar</code>	Keeps Scholar as a frontend deep link only.
<code>test_startup_migration_adds_note_columns_idempotently</code>	Keeps dev DB upgrades repeatable.

11. Performance optimizations

Optimization	Before	After or result
Deduplicator identifier indexes and gates	O(n squared) fuzzy compare across source fan-out.	Typical batch dedupe around 2 ms.
Hard per-source <code>asyncio.timeout</code>	httpx per-operation timeout let slow responses exceed the intended budget.	Search completes with degraded health, about 4 to 13 seconds depending on throttling.
ContextBuilder 1800-word budget	Oversized prompt could produce empty drafts.	Evidence remains within local model context.
Ollama embedding batches of 32	One request per chunk.	Fewer HTTP calls while preserving order.
Existing chunk reuse	Re-pinning could re-fetch and re-embed.	Reuses persisted chunks and vectors across spaces.
Query rewrite only for referential questions	Full memory/query blobs polluted retrieval.	Short standalone rewrite plus original query for fusion.

12. Design vs. implementation notes

The built system is intentionally simpler than the original design docs. There is no LangGraph, Celery, Redis worker, production Postgres, GROBID parser, or required Qdrant vector database. The actual code uses FastAPI services, SQLite, pypdf, in-process ingestion, an in-memory vector store rehydrated from SQLite embeddings, and a plain async verification loop. This is the behavior documented above and verified against the source code, live OpenAPI, and `dev.db` schema.